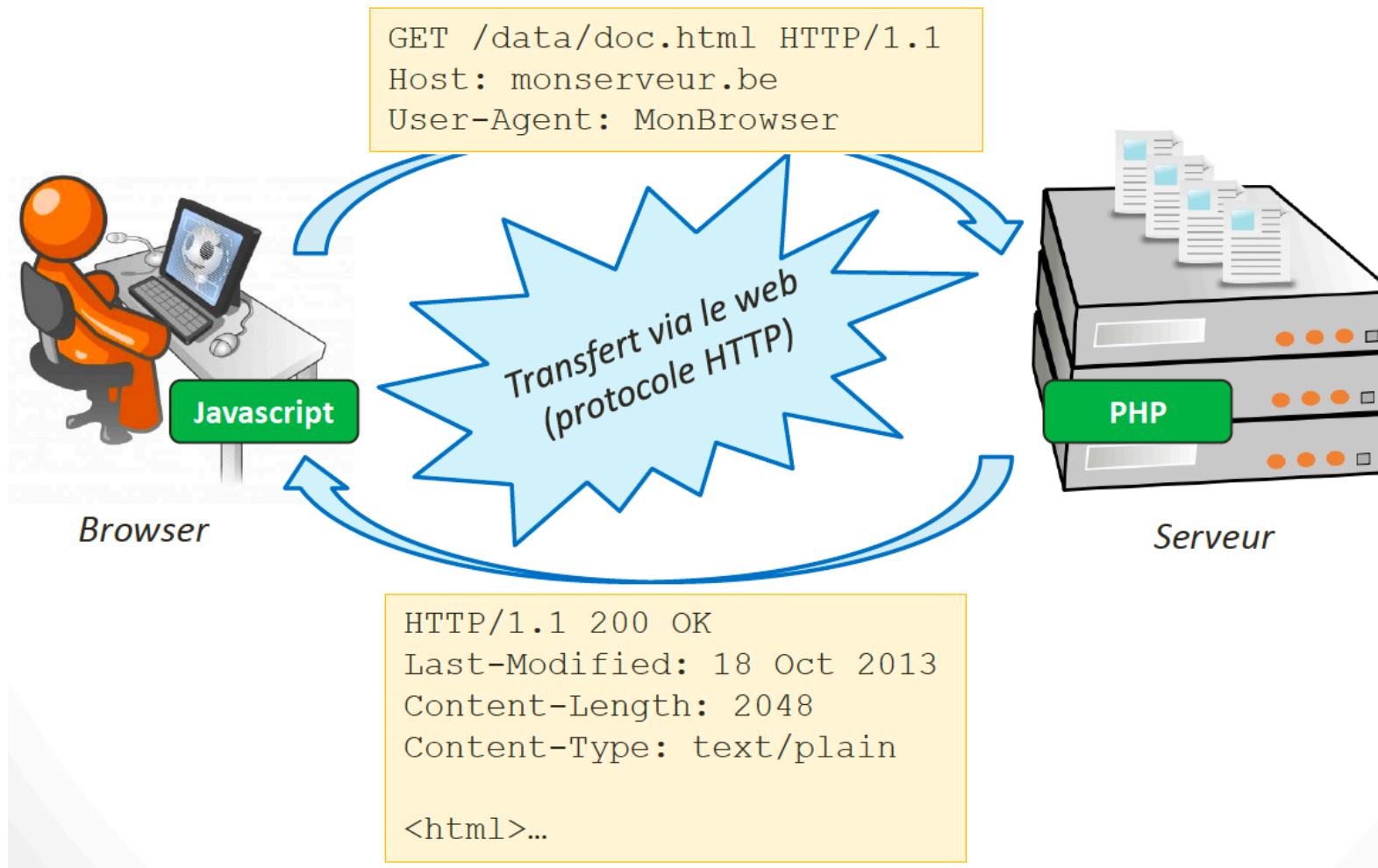

échange de données

On parle de quoi ?

- 1 L'échange de données en PHP
- 2 GET et POST
- 3 Cookies et sessions

L'échange de données en PHP



L'échange de données en PHP

Les informations envoyées au serveur sont souvent plus complexes qu'une simple URL du document désiré.

POST



"Je suis un admin, voici mon pseudo et un mot de passe"

GET



"La catégorie de vêtement qui m'intéresse est 'Pantalon'"

COOKIES



"C'est encore moi l'admin de tout à l'heure... souviens-toi de moi !"

L'échange de données en PHP

Les informations envoyées au client ne sont pas toujours simplement un document HTML entier

AJAX



Scroll infini, autocomplétion, etc.

L'échange de données en PHP



Un serveur PHP n'a pas de mémoire

STATELESS



Requêtes séparées

Chaque requête est traitée indépendamment



Oubli immédiat

Après traitement, tout est effacé de la mémoire



Aucune trace

Pas de souvenir des requêtes précédentes

Méthode GET

GET

Les informations se trouvent directement dans l'URL

 Format de l'URL

url

?

clé=valeur

&

clé=valeur

&

clé=valeur

Méthode GET

 Exemple concret

```
https://www.google.be/search?q=javascript
```

Méthode GET

Méthode GET - Côté client

💡 Les données peuvent être écrites de 3 façons

HTML



Lien direct

Données intégrées dans l'attribut href

```
href="monsite.be?cat=Pantalon"
```

SIMPLE

STATIQUE

Méthode GET - Côté client

💡 Les données peuvent être écrites de 3 façons

JS



Script direct

Redirection immédiate avec URL fixe

```
location.href ="monsite.be?cat=Pantalon";
```

IMMÉDIAT

PROGRAMMÉ

Méthode GET - Côté client



Les données peuvent être écrites de 3 façons

JS



Construction dynamique

URL construite à partir des données utilisateur

```
let cat = document.getElementById("cat").value;  
let url = "monsite.be?cat=" + cat;  
location.href = url;
```

DYNAMIQUE

INTERACTIF

Méthode GET - Côté serveur

✉ URL reçue

```
http://monsite.be/page.php?nom=Dupont&age=25
```

nom = Dupont

age = 25

Méthode GET - Côté serveur

Code PHP

```
if (isset($_GET['nom'])){  
    $nom = $_GET['nom']; // "Dupont"  
}  
  
if (isset($_GET['age'])){  
    $age = $_GET['age']; // "25"  
}
```

Méthode GET - Côté serveur

✓ Variables créées

\$nom

→

"Dupont"

\$age

→

"25"

Méthode GET - Côté serveur

🎯 Correspondance clé/valeur

nom = Dupont



```
$_GET['nom'] = "Dupont"
```

age = 25



Méthode GET - Encodage des URL

⚠ Caractères spéciaux dans les URLs

❌ Problème

`&nom=jules dupont`



✅ Encodé

`&nom=jules+dupont`

JS

`encodeURIComponent()`

Méthode POST

POST

Les informations sont transmises dans un paquet séparé, invisible dans l'URL

Méthode POST



Méthode POST - Côté client

 Formulaire HTML envoyé

Nom :

Âge :

Envoyer (POST)

nom = Dupont

age = 25

Méthode POST - Côté serveur

Code PHP

```
if (isset($_POST['nom'])){  
    $nom = $_POST['nom']; // "Dupont"  
}  
  
if (isset($_POST['age'])){  
    $age = $_POST['age']; // "25"  
}
```

Méthode POST - Côté serveur

🎯 Correspondance clé/valeur

nom = Dupont



```
$_POST['nom'] = "Dupont"
```

age = 25



```
$_POST['age'] = "25"
```

Cookies et sessions



Une application web est "stateless"

SANS MÉMOIRE



Pas de suivi utilisateur

Impossible de retenir le login ou le niveau d'accès automatiquement



Pas de mémoire

Aucun souvenir des données d'une page à l'autre

Cookies et sessions

1

Rends-toi sur un site de vente en ligne

2

Ajoute quelques articles dans le panier

3

Ferme le navigateur

4

Ouvre de nouveau le navigateur et retourne sur le site

✓

Ton panier contient toujours les articles !



Refais en navigation privée et observe la différence

Cookies et sessions

Le mécanisme de suivi utilisateur

Comment "suivre" (pister, tracker) un utilisateur ?



COOKIES

Stockage Client

Données stockées chez le client



Sur l'ordinateur de l'utilisateur



SESSION

Stockage Serveur

Données stockées sur le serveur



Sur le serveur web

Cookies



Caractéristiques des Cookies



Stockage local

Stockées sur le disque dur du client



Gestion navigateur

Écrites et lues par le navigateur



Liées au site

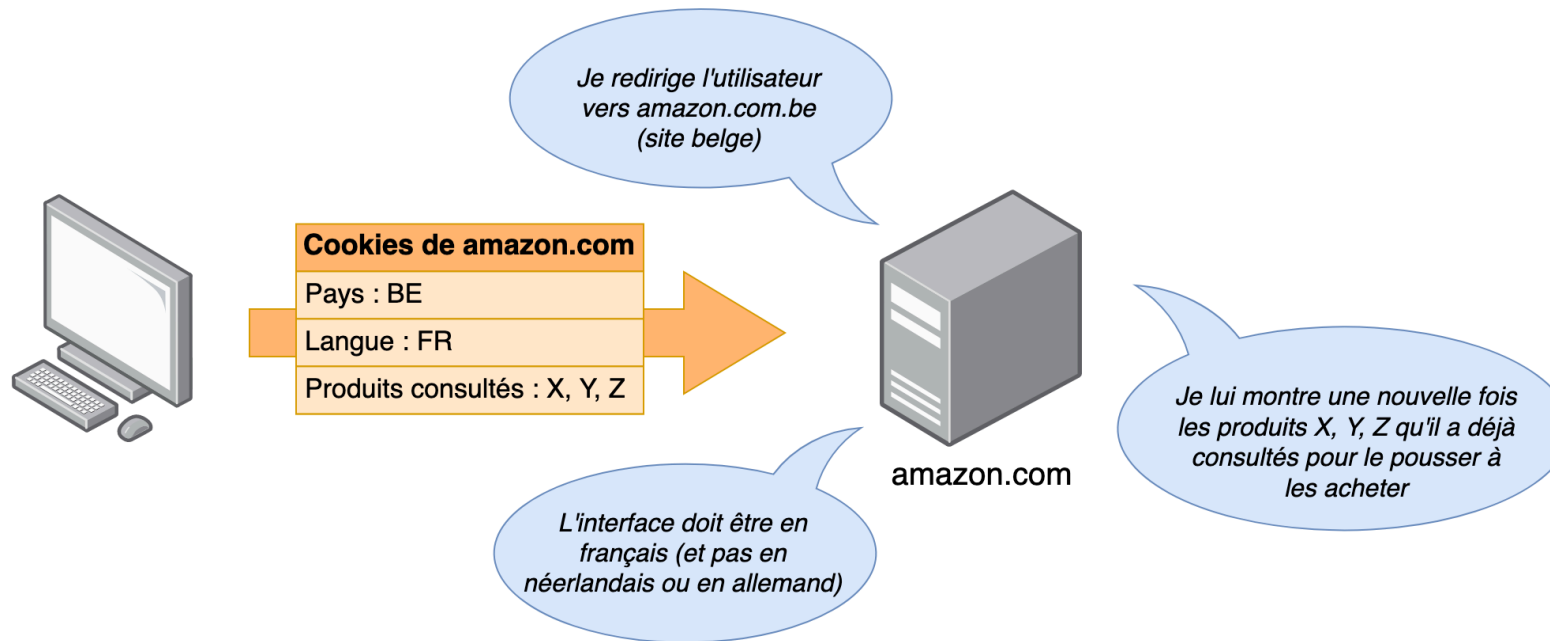
Associées à un domaine spécifique



Auto-envoi

Envoyées avec chaque requête au serveur

Cookies



Cookies - Manipulations



Définir / Modifier

```
// Délai d'expiration de 7 jours  
setcookie('clé', 'valeur', time() + 60*60*24*7);
```

Cookies - Manipulations



Accéder

```
if(isset($_COOKIE['clé'])){  
    echo $_COOKIE['clé'];  
}
```

Cookies - Manipulations



Détruire

```
// délai d'expiration dans le passé  
setcookie('clé', '', time() - 3600);
```

Cookies - ATTENTION !



POSITION CRITIQUE DE SETCOOKIE()

Les cookies doivent être définis **AVANT** toute sortie HTML
(même un espace ou retour à la ligne) !

Cookies - Position setcookie()

```
<?php
    if (isset($_POST['nom'])) {
        setcookie('nom', $_POST['nom'], time() + 60*60*24*7);
    }
?>
<!doctype HTML>
<html>
    <head>...</head>
    <body>...</body>
</html>
```



Le setcookie() doit être placé **avant** le DOCTYPE

Cookies - Pourquoi cette restriction ?



Explication technique

1

Les cookies sont envoyés dans les en-têtes HTTP

Ils font partie de la réponse du serveur au navigateur

2

Les en-têtes doivent être envoyés EN PREMIER

Avant le contenu HTML de la page

3

Une fois le HTML commencé, c'est trop tard !

Les en-têtes ont déjà été envoyés au navigateur

htmlspecialchars() - Protection XSS

Fonction PHP qui **échappe les caractères HTML spéciaux** pour éviter les injections XSS (Cross-Site Scripting).

Caractères échappés :

→ & → &

→ < → <

→ > → >

→ " → "

→ ' → '

```
echo "Bonjour " . htmlspecialchars($_COOKIE['nom']) . " !";
```

Cookies - Exemple complet

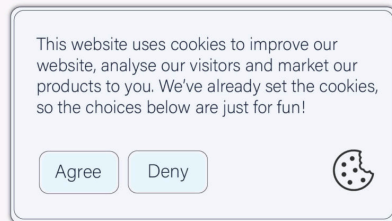
```
<?php
if (isset($_POST['nom'])) {
    setcookie('nom', $_POST['nom'], time() + 60*60*24*7);
}
?>
<!DOCTYPE html>
<html>
<body>
    <form method="POST" action="index.php">
        <input type="text" name="nom">
        <button type="submit">Enregistrer</button>
    </form>
    <?php
    if (isset($_COOKIE['nom'])) {
        echo "<p>Bonjour " . htmlspecialchars($_COOKIE['nom']) . " !</p>";
    } elseif (isset($_POST['nom'])) {
        echo "<p>Bonjour " . htmlspecialchars($_POST['nom']) . " !</p>";
    }
    ?>
</body>
</html>
```

Sessions



Problème

Et si le client interdit les cookies ?



Solution

Stocker les données sur le serveur, c'est plus sûr !



Sécurisé



Contrôlé



Fiable

Sessions - Fonctionnement

NEW Les données ne sont plus stockées chez le client !



amazon.com

Données de session

Pays : BE

Langue : FR

Produits consultés : X, Y, Z

Données de session

Pays : FR

Langue : FR

Produits consultés : A, B, C

Données de session

Pays : DE

Langue : DE

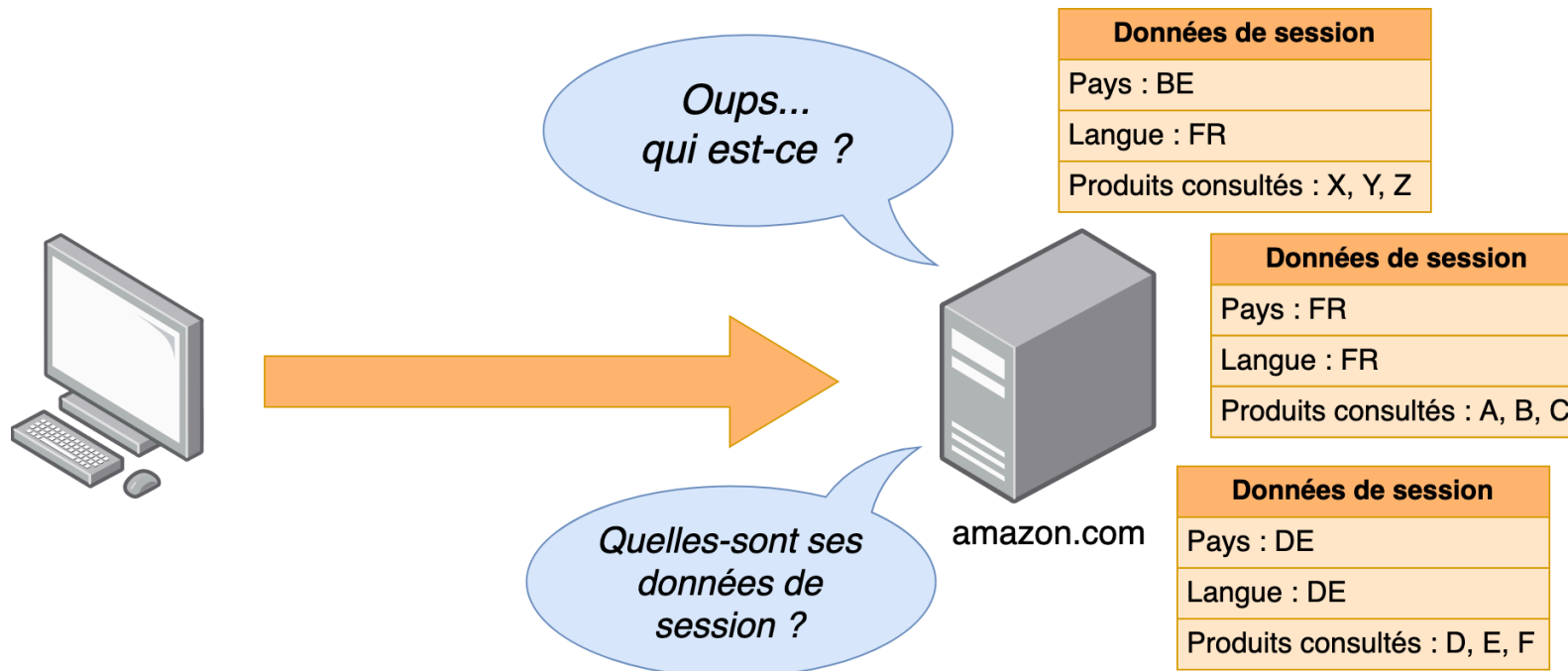
Produits consultés : D, E, F

Sessions - Fonctionnement



Comment savoir quelles données correspondent à quel client ?

Sessions - Fonctionnement



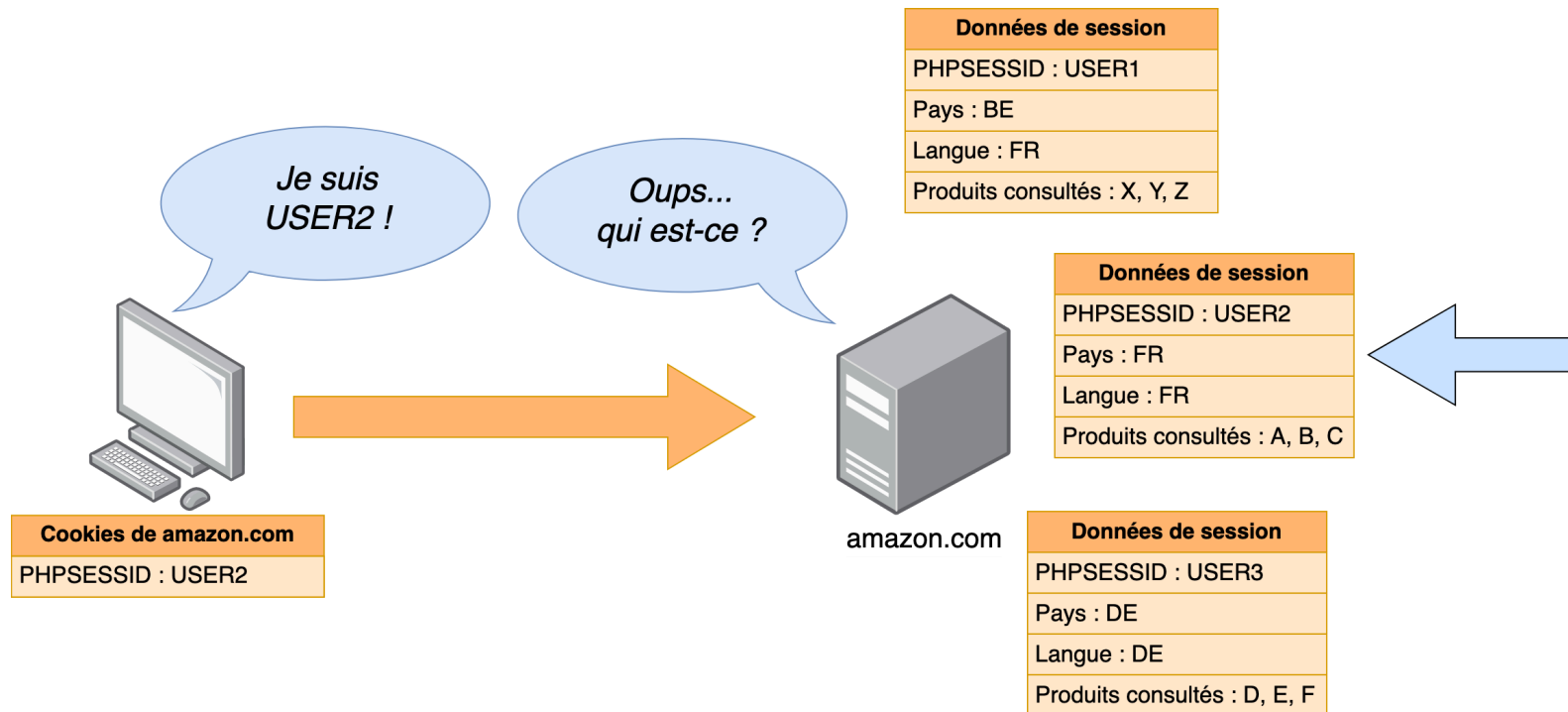
Sessions - Fonctionnement



Solution : attribuer un identifiant unique à chaque client

L'identifiant est stocké dans un cookie (PHPSESSID)

Sessions - Fonctionnement



Sessions - Remarque



Oui mais...

Et si le client interdit les cookies ?



Dans ce cas, les sessions ne fonctionneront pas car l'identifiant de session est stocké dans un cookie !

Sessions - Manipulation



Commencer une session

```
<?php
session_start();
?>
<!doctype HTML>
<html>
  <head>...</head>
  <body>...</body>
</html>
```

⚠ À exécuter en tout premier, avant tout HTML !

Sessions - Manipulation



Créer / Modifier une variable de session

```
<?php
$_SESSION['clé'] = 'valeur';
?>
```

Sessions - Manipulation



Lire une variable de session

```
<?php
if(isset($_SESSION['clé'])){
    echo $_SESSION['clé'];
}
?>
```

Sessions - Manipulation



Supprimer une variable de session

```
<?php  
unset($_SESSION['clé']);  
?>
```

Sessions - Manipulation



Terminer / Effacer une session

```
<?php
session_destroy();
$_SESSION = array();
?>
```

💡 *Détruit toutes les données de session*

💡 *Cas classique : l'utilisateur se déconnecte*

Sessions - Exemple complet

```
<?php
session_start();
if (isset($_POST['nom'])) {
    $_SESSION['nom'] = $_POST['nom'];
}
if (isset($_POST['logout'])) {
    session_destroy();
    header("Location: index.php");
    exit();
}
?>
<!DOCTYPE html>
<html>
<body>
    <?php
    if (isset($_SESSION['nom'])) {
        echo "<p>Bonjour " . $_SESSION['nom'] . " !</p>";
        echo '<form method="POST" action="index.php">';
        echo '<button type="submit" name="logout">Se déconnecter</button>';
        echo '</form>';
    }
    else {
        echo '<form method="POST" action="index.php">';
        echo '<input type="text" name="nom" placeholder="Votre nom">';
        echo '<button type="submit">Enregistrer</button>';
        echo '</form>';
    }
    ?>
</body>
</html>
```


Cookies vs. Sessions

Cookies	Sessions
Stockage chez le client	Stockage sur le serveur
Pas de risque de surcharge	Risque de surcharge (puissance, stockage)
Peu de sécurité	Sécurité plus grande
Utilisés pour le stockage de longue durée	Utilisé pour le stockage de courte durée
Exemple : choix de la langue	Exemple : données de panier